Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

A Quantum Neural Network for Fraud Detection Using a Data-Driven Priority Entanglement Scheme

Yousaf Khaliq

Middle Tennessee State University

Donglin Wang

dwang@mtsu.edu

Middle Tennessee State University

Research Article

Keywords: Quantum Machine Learning, Quantum Neural Network Classifier, Fraud Detection, Custom Entanglement

Posted Date: November 18th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-7991330/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

A Quantum Neural Network for Fraud Detection Using a Data-Driven Priority Entanglement Scheme

Yousaf Khaliq[1] and Donglin Wang[2*]

[1]Department of Computer Science, Computational and Data Science Program, Middle Tennessee State University, 1301 E Main St, Murfreesboro, 37132, TN, United States.

[2*]Department of Mathematical Sciences, Computational and Data Science Program, Middle Tennessee State University, 1301 E Main St, Murfreesboro, 37132, TN, United States.

*Corresponding author(s). E-mail(s): dwang@mtsu.edu;
Contributing authors: yak2a@mtmail.mtsu.edu;

Abstract

This study investigates whether a variational quantum circuit can improve minority-class sensitivity when substituted for a single dense layer in a strong neural network baseline for credit-card fraud detection. Under matched parameter budgets and identical preprocessing, we evaluate a hybrid quantum–classical model against classical baselines on the Kaggle dataset, preserving all 492 fraud cases and downsampling legitimate transactions to 10,000. Our hybrid employs data re-uploading to expose all 30 features with 10–15 qubits. It introduces a data-driven priority-entanglement scheme that couples the most dependent feature pairs before a strongly entangling block. Across 100 (10-qubit) and 20 (15-qubit) randomized runs, the hybrid achieves meaningful and consistent gains in recall and PR-AUC over tuned classical models (e.g., PR-AUC 0.9229±0.006 at 15q/1-layer/10 priority pairs vs 0.9150±0.014 for logistic regression; recall +0.047 over the best classical result).Performance peaks at moderate entanglement budgets, beyond which deeper circuits only exacerbate a precision-recall trade-off, revealing an underlying trainability limit. Results indicate that correlation-guided entanglement provides a useful inductive bias that modestly improves fraud detection under strict capacity parity.

Keywords: Quantum Machine Learning, Quantum Neural Network Classifier, Fraud Detection, Custom Entanglement

1 Introduction

Credit card fraud imposes significant costs on issuers, merchants, and cardholders, and fraud detection is a critical challenge for financial institutions. Practical detection systems must operate under severe class imbalance, changing attacker behavior, and tight latency budgets. In production, authorization decisions are made within tens of milliseconds, which limits model size, feature lookups, and heavy ensembling. A missed fraud becomes a chargeback, while a false alarm blocks a legitimate purchase and hurts the cardholder experience. These operational constraints motivate lightweight, yet expressive, models capable of adapting to evolving data distributions. Regarding attack behaviors, fraud patterns are non-stationary—card testing, merchant takeovers, and coordinated mule activity induce distribution shifts, so models require robust evaluation, drift monitoring, and frequent recalibration.

The publicly available Kaggle Credit Card Fraud dataset (Machine Learning Group, ULB and Worldline, 2019) has become a canonical benchmark for these constraints: 284,807 transactions, of which 492 ($\sim 0.17\%$) are labeled fraudulent, with features anonymized and reduced to 30 by principal component analysis as documented by (Dornadula and Geetha, 2019; Itoo et al., 2021) and (Qi, 2025). In such settings, we value the recall over precision as the most valuable metric, followed by the precision-recall area under the curve (PR-AUC). Recall is the fraction of fraudulent transactions correctly identified; precision is the fraction of flagged transactions that are truly fraudulent (false alarms reduce precision). Precision is important; however, we want to capture as many fraudulent transactions as we can. A single percentage increase in recall can translate into millions in savings for a financial institution. These metrics are more informative than accuracy, Receiver Operating Characteristic Curve (ROC–AUC), or F1-score following guidance of (Dornadula and Geetha, 2019) and (Itoo et al., 2021). Models can easily learn to classify every transaction as normal, which leads to an accuracy of $\sim 99.8\%$. Classical machine learning methods, such as ensemble methods, support vector machines (SVMs) (Hearst et al., 1998), and neural network classifiers, remain strong in this benchmark, but performance often hinges on careful handling of imbalances (Qi, 2025). Even after handling imbalances, whether by synthetic oversampling using the synthetic minority oversampling technique (SMOTE) (Chawla et al., 2002) or rebalancing, there is always a limitation to find high-dimensional feature relationships and patterns within the data. A quantum neural network has the ability to glean rich, deeper, and more meaningful patterns within the data, especially a hybrid quantum classical model that combines the power of dense classical layers with a variational quantum circuit.

This work asks a narrow question: if we keep the architecture and parameter budget fixed, does swapping a single classical layer of a neural network classifier for a simulated variational quantum circuit (VQC) improve minority-class sensitivity on this benchmark? In other words, can a quantum-inspired neural network classifier achieve better recall and PR-AUC than its classical counterpart? To answer this question, we construct the fairest and most equitable comparison between a classical neural network classifier and a hybrid quantum-classical counterpart that replaces a dense layer with a end-to-end-trained VQC, following (Benedetti et al., 2019) and the overviews

by (Biamonte et al., 2017) and (Schuld and Petruccione, 2018). To expose all 30 features with a limited qubit simulation budget, we use data reuploading in either three 10-qubit chunks or two 15-qubit chunks, following the data reuploading strategy of (Pérez-Salinas et al., 2020).

We also introduce a simple, data-driven priority entanglement scheme: compute the Pearson correlation matrix over features and allocate two-qubit couplings first to the most strongly correlated feature pairs; a final, trainable strongly entangled block then propagates information globally. Because full-state quantum simulation is computationally expensive, we train on a downsampled set of 10,000 legitimate transactions while preserving all 492 frauds. Under identical preprocessing, optimizer, batch size, epochs, and comparable parameter counts, we run 100 randomized trials and report the recall, precision, and PR-AUC. Across the 100 runs, the proposed hybrid model with correlation-aware entanglement improves average Recall and PR-AUC relative to the classical counterpart under matched budgets, suggesting that re-uploading plus structured entanglement can tilt the precision-recall tradeoff in favor of minority detection on this benchmark.

Our main contributions are: (i) a correlation-guided custom entanglement scheme that leverages feature dependencies; (ii) a controlled hybrid-versus-classical benchmark under matched parameter budgets; and (iii) a 100-run evaluation emphasizing recall and PR-AUC under extreme imbalance. Through these contributions, our experiment was able to make meaningful discoveries regarding the effect of boosting or hindering the quantum component's power and the effect of the novel priority pair entanglement scheme. Among those discoveries include finding that 15 qubits with 3 or more quantum layers (split into two chunks via re-uploading) improved recall; however, at the cost of precision. At the same time, models run with 30 entanglement pairs tended to achieve the best recall and PR-AUC. This was interesting because there was a clear diminished return as we increased the number of priority pairs beyond 30, which we believe is due to over-parameterization. We provide exact values and figures with an in-depth analysis in our results and discussion sections.

The remainder of this paper is organized as follows: Section 2 reviews related work, Section 3 describes the proposed methods, Section 4 presents results, Section 5 discusses implications, and Section 6 outlines future directions and concludes this research.

2 Background and Related Work

2.1 Fraud detection on the Kaggle benchmark

Methodologies applied to the Kaggle dataset span a wide spectrum, including Logistic Regression (Nick and Campbell, 2007), Support Vector Machines (Hearst et al., 1998), ensemble methods such as Random Forest (Rigatti, 2017) and XGBoost (Chen et al., 2015), as well as classical deep neural networks (Rumelhart et al., 1986).

These methods typically combine feature engineering, supervised learning, and post-hoc thresholding. With the positive class being so imbalanced, the evaluation emphasizes the PR-AUC, recall, and expected cost curves. The Kaggle dataset provides anonymized, principal component analysis (PCA)-like features over two days

of European card transactions and is now a standard benchmark. Standard practice includes stratified validation, leakage checks, calibration, and careful class-imbalance handling. SMOTE is most commonly employed to synthetically increase the number of fraudulent transactions to train and test while keeping the entirety of the legitimate transactions (Qi, 2025). It is understandable to approach the problem in this way; however, this can often create biased results as test sets become "touched", thereby unfairly increasing the models' generalization ability. Our experiment does the opposite. To preserve the natural fraud distribution, we instead retain all 492 frauds and downsample 10,000 legitimate transactions—ensuring that the minority class remains untouched and more representative of production conditions. Additionally, we attach a heavier weight to missed fraudulent transactions with binary cross-entropy with logits to further address the class imbalance. This is one of the main differences between our experiment and previously conducted research on this particular Kaggle dataset.

2.2 Quantum background essentials

To access the computational ability of quantum machine learning we must begin with the qubit, which is an evolution of the classical binary bit. A single-qubit is a unit vector in a two–dimensional Hilbert space with computational basis $\{|0\rangle, |1\rangle\}$. A pure state can be written

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle, \qquad |\alpha|^2 + |\beta|^2 = 1.$$

Equivalently on the Bloch sphere,

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\phi}\sin\frac{\theta}{2}\,|1\rangle, \quad \theta \in [0,\pi],\ \phi \in [0, 2\pi).$$

For n qubits the joint space is $\mathbb{C}^{2^n}$ with basis $\{|b_{n-1}\cdots b_0\rangle : \ b_k \in \{0,1\}\}$ (Nielsen and Chuang, 2010).

Two qubits can then exhibit a quantum phenomenon known as entanglement. A bipartite state $|\Psi\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^2$ is *separable* if $|\Psi\rangle = |\psi\rangle \otimes |\phi\rangle$ for some single–qubit states; otherwise it is entangled. Entangled qubits have outcomes that directly effect each other; we leverage this quantum phenomenon heavily in our experiment when processing our data (Nielsen and Chuang, 2010).

We use quantum gates to manipulate our qubits as we see fit, just as classical logic gates manipulate classical bits. Quantum gates are unitary operators $U \in \mathrm{U}(2^n)$ and are reversible transformations making them highly suitable for quantum computation and simulation. Common single–qubit gates are the Pauli and Hadamard matrices

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

One of the most important quantum gates we will leverage is the rotation gate. Parameterized single–qubit rotations about Pauli axes are

$$R_X(\theta) = e^{-i\theta X/2} = \cos\frac{\theta}{2}\, I - i\sin\frac{\theta}{2}\, X, \quad R_Y(\theta) = e^{-i\theta Y/2} = \cos\frac{\theta}{2}\, I - i\sin\frac{\theta}{2}\, Y,$$

$$R_Z(\theta) = e^{-i\theta Z/2} = \cos\frac{\theta}{2}\, I - i\sin\frac{\theta}{2}\, Z.$$

These define continuous families used for both data encoding and trainable variational parameters (Nielsen and Chuang, 2010).

The second and equally important quantum gate is the two–qubit controlled–NOT. The CNOT gate acts as

$$\mathrm{CNOT}\,|a, b\rangle = |a,\, a \oplus b\rangle, \qquad a, b \in \{0, 1\},$$

and is essentially how we entangle two qubits. For our purposes, we entangle qubits that have been mapped to features of our dataset. This allows for a global information flow and feature interaction computation in a high-dimensional, complex space (Barenco et al., 1995).

Measurements of the qubits are then conducted after manipulation via gates. Projective measurement in the computational basis $\{\Pi_0 = |0\rangle\langle 0|,\ \Pi_1 = |1\rangle\langle 1|\}$ yields

$$\Pr[0] = \langle\psi|\Pi_0|\psi\rangle = |\alpha|^2, \quad \Pr[1] = |\beta|^2.$$

More generally, for an observable M (Hermitian), the expectation under a pure state $|\psi\rangle$ or density operator ρ is

$$\langle M\rangle = \langle\psi|M|\psi\rangle = \mathrm{Tr}(M\rho).$$

Variational classifiers typically measure tensor–products of Pauli operators (e.g., $M = Z^{\otimes k} \otimes I^{\otimes(n-k)}$) and use $\langle M\rangle \in [-1, 1]$ as classical features (Nielsen and Chuang, 2010; Benedetti et al., 2019).

2.3 Variational quantum circuits (tying it all together)

Our work combines the above stated quantum background essentials to create the variational quantum circuit (VQC). VQCs offer a new way to process information especially when integrated with traditional neural networks; they can be parameterized and trained via rotation layers and entanglers. In hybrid setups, these circuits can act as a layer that maps classical features to expectation values, which are then fed back into a classical network. We follow this view and swap a single dense layer with a VQC keeping the rest of the pipeline fixed, in line with (Benedetti et al., 2019) and reviewed by (Cerezo et al., 2021).

A variational circuit composes data–encoding rotations and trainable entangling blocks. Let $x \in \mathbb{R}^d$ denote classical features and θ the trainable parameters. A common

ansatz is

$$U(x,\theta) = \prod_{\ell=1}^{L} \Big(E_\ell(\theta_\ell)\, R(x_{\sigma(\ell)}) \Big),$$

where $R(\cdot)$ applies single–qubit rotations (e.g., R_Y, R_Z) to encode components of x and E_ℓ is a pattern of entangling gates (e.g., CNOTs). Given an initial state $|0\rangle^{\otimes n}$ and an observable M (often a tensor of Z's), the circuit outputs the expectation

$$f(x;\theta) = \langle 0^{\otimes n} |\, U^\dagger(x,\theta)\, M\, U(x,\theta)\, |0^{\otimes n}\rangle \in [-1,1],$$

which serves as the (differentiable) quantum layer's output fed to a classical head. In this work, we use rotations for data re–uploading, CNOT–based entanglers (both priority pairs and strongly entangling layers), and Pauli–Z measurements to produce a Q–dimensional feature vector from Q qubits.

2.4 Data re-uploading

Re-uploading repeatedly injects classical features into the circuit across layers, boosting expressivity with few qubits. (Pérez-Salinas et al., 2020) shows universal-approximation style results with re-uploading, and multi-qubit extensions benefit further from entanglement. We use re-uploading to cycle all 30 features through $Q \in \{10, 15\}$ qubits in $C = \lceil 30/Q \rceil$ chunks (three for Q=10, two for Q=15).

Re-uploading, in our context, can be seen as a partition of $x \in \mathbb{R}^{30}$ into $C = \lceil 30/Q \rceil$ chunks $x^{(c)}$

$$R\big(x^{(c)}\big) = \bigotimes_{q=1}^{Q} R_Y\big(\alpha\, x_q^{(c)}\big)\, R_Z\big(\beta\, x_q^{(c)}\big),$$

and a depth-L layer stacks $[E_\ell(\theta_\ell) R(x^{(c)})]$ for $c = 1, \ldots, C$.

2.5 Priority entanglement scheme

Entanglement patterns control how local feature information mixes. Common patterns (linear, ring, strongly entangling layers) provide broad mixing, but do not exploit the structure of the dataset, as cataloged by (Benedetti et al., 2019). Our priority entanglement first couples feature pairs with the highest empirical Pearson correlations, then apply a strongly entangling block to distribute correlations globally. Intuitively, early layers bias toward empirically coupled features; later layers redistribute information more broadly.

Let $\rho \in \mathbb{R}^{30\times 30}$ be the Pearson matrix in train data. Select k feature pairs by

$$\mathcal{P}_k = \arg\operatorname*{top\text{-}k}_{i<j}\ |\rho_{ij}|,$$

map features $\phi(i) \mapsto q(i)$ to qubits, then realize $\mathcal{E}_k = \{\, \mathrm{CNOT}(q(i), q(j)) : (i,j) \in \mathcal{P}_k \,\}$ before a block that strongly entanglements.

The model's loss landscape exists in a complex Hilbert space of exponential dimensions and can often lead to barren plateaus, vanishing gradients, local minima, etc.

(McClean et al., 2018). Another way to mitigate these issues is by implementing more shallow circuits as conducted by (Cerezo et al., 2021), however, we wish to utilize as many qubits as computationally feasible in order to encompass as much data representation as possible across all 30 features. A circuit with fewer qubits would not grant us access to the VQC's full representational potential. Our intention behind the priority entanglement scheme with data re-uploading is to provide a "helping hand" to the model by guiding the optimization and keeping as much quantum power as we can. In addition, replacing only one dense classical layer with the VQC allows for a clean isolation of the quantum component's contribution.

2.6 QML in finance and fraud

Finance has been a testbed for quantum machine learning (QML) in portfolio optimization, option pricing, risk analysis, and anomaly detection, as surveyed by (Grossi et al., 2022) and (Egger et al., 2020), with applications to risk (Wilkens and Moorhouse, 2023) and forecasting (Thakkar et al., 2024). For the Kaggle fraud dataset specifically, prior work includes quantum kernels (QSVM) (Akrom, 2024; Park et al., 2020), VQC classifiers, and quantum graph neural networks (QGNNs) by (Innan et al., 2024). Reported results vary and comparisons to tuned classical baselines under matched budgets are limited. We position our study as a controlled, thorough examination of a high performing classical baseline versus its quantum counterpart evaluated under repeated trials on the same reliable, indicative benchmarks. Additionally, our work is not an introduction of a new quantum model, but rather an innovation in how an existing quantum model processes information (the priority pair entanglement scheme) followed by a thorough and rigorous experiment testing said innovation.

2.7 Trainability, data, and design considerations

Trainability challenges such as barren plateaus motivate careful choices of observables, depth, and entanglement, following (McClean et al., 2018; Cerezo et al., 2021), and entanglement-induced barren plateaus discussed by (Ortiz Marrero et al., 2021); see also (Cerezo et al., 2022) for broader challenges. Data-limited regimes (such as strong downsampling) also affect generalization and may favor compact models, as argued by (Caro et al., 2022) and (Huang et al., 2021). Finally, real hardware implementation introduces noise, limited connectivity, and runtime constraints, which are not addressed in this simulation study but are crucial directions for future exploration.

We choose a hybrid layer simply because it enforces parity with a strong classical baseline. We (i) keep preprocessing identical, (ii) fix optimizer, schedule, batch size, and epochs, (iii) swap exactly one dense layer for a VQC with re-uploading, and (iv) downsample only the majority class (to 10,000) while keeping all fraud cases (492). We then run 100 seeded trials and ablate re-uploading depth, qubit budget, and entanglement policy.

2.8 Scope and limitations

Our experiments run on a simulator; hardware noise/connectivity may alter outcomes and most certainly increase runtimes drastically. Furthermore, the anonymized PCA-like features limit semantic analysis (Abdi and Williams, 2010). We mitigate variance via 100 seeds, fixed budgets, and ablations (qubit count, quantum layer count, and priority pair count), but broader claims require additional datasets and eventual hardware tests. Foundational overviews of quantum machine learning and ensembles further motivate our design choices but also underline open questions about advantage under realistic constraints. Future work will extend the proposed entanglement design to hardware implementations and investigate robustness under noisy conditions.

3 Methodology

3.1 Overview

We benchmark a parity-matched classical classifier against a hybrid quantum–classical variant on the Kaggle credit card fraud task. Both models share identical feature-extraction trunk and logistic head; the only difference is the middle block: a dense layer for the classical baseline versus a variational quantum layer for the hybrid. This isolates the contribution of the quantum component under a fixed parameter budget and common training protocol.

We evaluated a hybrid quantum-classical classifier against an evenly matched classical neural network on the Kaggle credit card fraud dataset. In this section, we outline our foundational data science approach to this problem, the hybrid and classical architectures, and the inner workings of the variational quantum circuit including our priority pair entanglement scheme. We then collect the recall, precision, PR-AUC, and CPU time over the average of 100 runs and share them in Section 4. The components in each experiment are controlled from a single configuration dictionary (hereafter referred to as CONFIG); ablations change only one component at a time i.e. 10 or 15 qubits, 1, 3, or 5 quantum layers, and 0, 10, 30, 80, or 135 priority pairs.

3.2 Data and preprocessing

Minimal preprocessing was necessary because PCA has already been applied to the dataset.

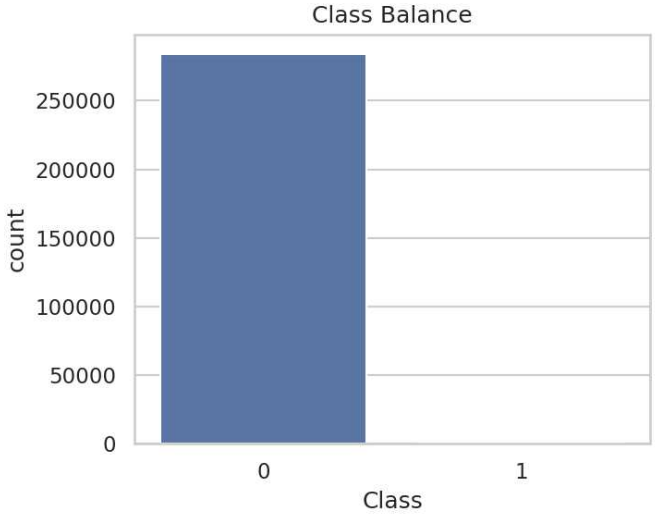

Fig. 1: Kaggle credit card fraud detection dataset with extreme class imbalance

The dataset consists of 284,315 legitimate transactions and 492 fraudulent transactions (the minority class) (Machine Learning Group, ULB and Worldline, 2019). Figure 1 illustrates the extreme class imbalance. After PCA, the dataset contains 30 features including `Time` and `Amount`. We downsample the majority class to `sample_size` (10,000) and keep all 492 fraud cases. We add an `hour_of_the_day` feature from the raw `Time` column via (`Time`/3600) mod 24, then drop `Time`.

We stratify into training, validation, and test sets with a split of 50/25/25. This results in a test set of 2500 legitimate transactions and 123 fraudulent. The validation set is used to tune our model, while the test set is completely "untouched" and is used to properly evaluate our model. All splits use `random_state=42` and are stratified by the label to preserve the base rates.

A `StandardScaler` from python library`scikit-learn` (Pedregosa et al., 2011) is fitted on the *training features only* and applied to the validation and test sets. We save the scaler (`scaler.pkl`) and the final priority pair feature order for reproducibility.

3.3 Train–only correlation analysis, feature reordering, and mapping

Let $X_{\text{train}} \in \mathbb{R}^{n \times d}$ be the training features after the specified split. To inject data structure into entanglement, we calculate the absolute Pearson correlation matrix $\rho \in \mathbb{R}^{d \times d}$ on *train only* to avoid leakage and keep the test set "untouched", sort the upper–triangle pairs by $|\rho_{ij}|$, and greedily build a feature order by taking disjoint top pairs first and then appending remaining features. In order for the priority pairs to be entangled, they have to be in the same chunk (C), so reordering the features in the dataset is necessary. This order is applied (identically) to train/validation/test.

With $Q = \texttt{n_qubits}$ and re–uploading in $C = \lceil d/Q \rceil$ chunks, feature i occupies a chunk $c(i) = \lfloor i/Q \rfloor$ and qubit $q(i) = i \bmod Q$. We select up to `num_priority_pairs` disjoint pairs (i, j) with largest $|\rho_{ij}|$ *subject to* $c(i) = c(j)$ (so the 2–qubit gate is realizable within a chunk) and a per–feature degree cap. The selected set is stored and exported as `priority_entanglement_pairs.csv`.

Pseudo–code (priority entanglement)

1. Compute $\rho = |\mathrm{corr}(X_{\mathrm{train}})|$; list the upper triangle pairs in descending order $|\rho_{ij}|$.
2. Initialize degree counters $\deg(i) \leftarrow 0$ and an empty set $\mathcal{P}$.
3. For each pair (i, j) in order: if $c(i) = c(j)$ and $\deg(i), \deg(j) <$ cap and $|\mathcal{P}| <$ `num_priority_pairs`, then add (i, j) to $\mathcal{P}$ and increase degrees.
4. Store $\mathcal{P}$ grouped by chunk and the induced feature order.

3.4 Models

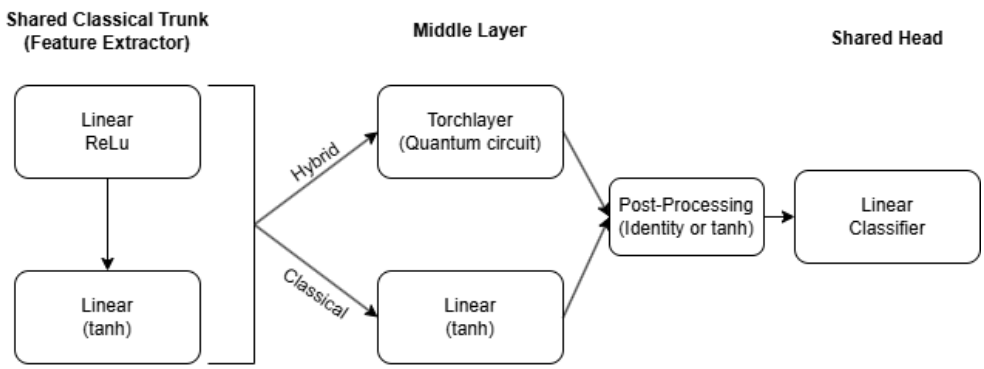

Fig. 2: Model Architecture

Figure 2 illustrates the shared and interchangeable architecture of our experiment.

Both models use an identical feature extractor via the shared classical trunk:

$$\mathrm{Trunk}(x) = \tanh\big(W_2\,\mathrm{ReLU}(W_1 x)\big), \qquad W_1, W_2 \in \mathbb{R}^{d\times d}.$$

The interchangeable middle layer can then be either (i) *Hybrid*: a quantum layer returning Q expectation values in $[-1, 1]$ via a PennyLane (Bergholm et al., 2018) `TorchLayer` or (ii) *Classical*: a linear map $\mathbb{R}^d \to \mathbb{R}^Q$ followed by tanh to match the hybrid's output range.

Lastly, a shared classifier head $h(z) = w^\top z + b$ outputs a logit; the predicted probability is $\sigma(h) = 1/(1 + e^{-h})$ and ultimately classifies a transaction from the dataset as legitimate or fraudulent.

3.5 Quantum circuit ansatz and readout

We simulate a Q–qubit device (Pennylane's `default.qubit`) and build a differentiable QNode with PyTorch interface and `diff_method=''backprop''`. For chunk index $c = 0, \ldots, C-1$:

1. **Angle embedding:** load the length–Q feature chunk with R_Y rotations (zero–pad if needed).
2. **Priority entanglers:** for each stored pair (q_i, q_j) with `chunk`$= c$, apply `CNOT`$(q_i \to q_j)$.
3. **Trainable block:** apply a strongly entangling layer (SEL) parameterized by $\theta[c] \in \mathbb{R}^{L_q \times Q \times 3}$, where $L_q =$ `num_q_layers`.

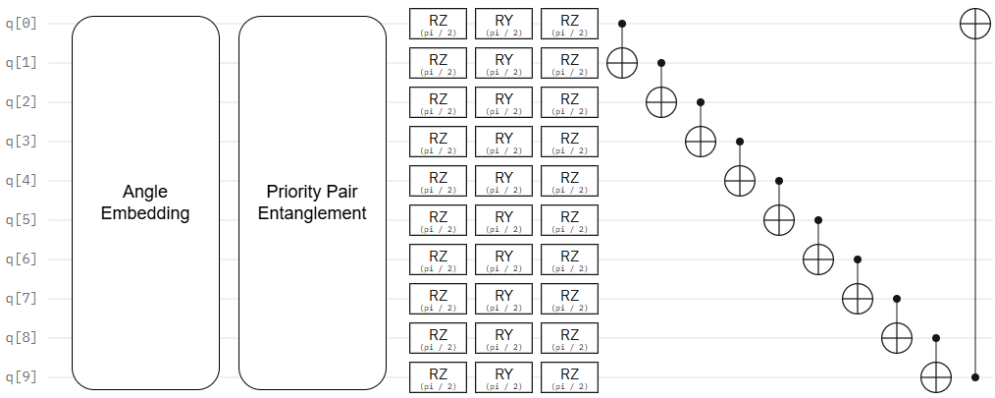

Fig. 3: A 10 qubit VQC with angle embedding followed by our priority pair entanglement scheme and then the strongly entangling layer

The SEL consists of three parameterized rotation gates and a ring of controlled-NOT gates to simulate global entanglement. The priority entanglers capture local empirical dependencies, whereas the SEL spreads information globally, ensuring expressive coverage under limited depth. Finally, we measure $\mathrm{E}[PauliZ_q]$ on each wire $q \in \{0, \ldots, Q-1\}$, which produces a Q–vector fed to the shared classifier head. These three components make up the entire circuit of our variational quantum, as seen in Figure 3, which is the middle layer implemented by PyTorch in our hybrid model.

3.6 Training protocol

We optimize binary cross–entropy with logits,

$$\mathcal{L} = -\, w_+ \, y \, \log \sigma(h) - (1-y) \, \log(1-\sigma(h)), \qquad w_+ = \frac{N_0}{N_1},$$

where w_+ balances the classes using the training set counts (N_1, N_0). We use the Adam optimizer (Kinga et al., 2015) with a learning rate of .001, batch size 32, and

training for 10 epochs. We monitor training/validation loss and accuracy each epoch (using a temporary zero threshold on logits for speed); final metrics are computed for the holdout test set.

3.7 Evaluation metrics, reporting, and ablations

From test set probabilities we generate a classification report and confusion matrix that computes Accuracy, Precision, Recall, F1, ROC–AUC, and PR–AUC. The threshold metrics use a configurable decision threshold (default `threshold`=0.3) chosen to tilt toward recall under extreme imbalance.

Unless otherwise noted, we vary one knob at a time while holding the rest fixed: number of qubits $Q \in \{10, 15\}$, quantum depth L_q, and the entanglement budget `num_priority_pairs`. We also probe sensitivity to `sample_size`, split fractions, learning rate, and the decision threshold.

3.8 Repetition, logging, and artifacts

Each 10-qubit configuration is repeated over 100 randomized runs and 15-qubit over 20 (distinct seeds for initialization and minibatch order; data sampling/splits are held fixed unless explicitly ablated). Every run writes to a timestamped directory under `outputs/` containing the active `config.txt`, learned weights for both models, the scaler (`scaler.pkl`), the feature order (`feature_order.pkl`), and, when nonempty, the entanglement pairs CSV. A master CSV log appends per-model precision, recall, and PR–AUC alongside configuration values for longitudinal comparison.

4 Results

4.1 Classical baselines

Table 1 summarizes four tuned classical models on the reduced dataset (10,000 normal + 492 fraud). The logistic regression and XGBoost achieve the highest precision (0.939±0.074 and 0.939±0.021), consistent with conservative decision boundaries, and yield PR-AUCs of 0.9150±0.014 and 0.9144±0.017, respectively. The classical neural network attains the strongest recall among the baselines (0.8832±0.014) but a lower PR-AUC (0.9036±0.008). These results set a competitive reference: high precision from linear/tree models and higher recall from the NN, all in the 0.91–0.92 PR-AUC band.

Model	Precision	Recall	PR-AUC
Classical Neural Network	0.65859 ± 0.072	**0.88324 ± 0.014**	0.9036 ± 0.008
Logistic Regression	0.93886 ± 0.074	0.85903 ± 0.021	**0.9150 ± 0.014**
XGBoost	**0.93921 ± 0.021**	0.87283 ± 0.030	0.9144 ± 0.017
Support Vector Machine	0.92759 ± 0.021	0.85872 ± 0.041	0.9120 ± 0.013

Table 1: Average ± standard deviation precision, recall, and PR-AUC over 100 runs for four traditional classifiers

4.2 Hybrid QNN (10 qubits): effect of entanglement and depth

Table 2 shows that with 10 qubits and a single quantum layer, introducing correlation-guided entanglement improves recall and PR-AUC over the no-pair baseline. The best PR-AUC occurs for 80 pairs (0.9192±0.008), while the best recall occurs at 135 pairs (0.9235±0.017). Precision generally declines as the number of priority pairs grows (e.g., 0.596 at 0 pairs → 0.561 at 135 pairs), reflecting the intended tilt toward minority-class sensitivity.

Increasing the quantum circuit depth from one to three or five layers does not lead to any improvement in PR–AUC at 10 qubits. In fact, the strongest PR-AUCs for 3- and 5-layer configurations are at or below the 1-layer case (e.g., 3 layers: 0.9162±0.010 at 135 pairs; 5 layers: 0.9162±0.012 at 0 pairs). This suggests diminishing returns and potential trainability costs from additional depth for this qubit budget.

10 qubits	0 pairs	10 pairs	30 pairs	80 pairs	135 pairs
1 qlayer	P: 0.59639 ± 0.068 R: 0.90869 ± 0.017 AUC: 0.9172 ± 0.007 CPU Time: 9.0m	P: 0.59702 ± 0.076 R: 0.90691 ± 0.018 AUC: 0.9172 ± 0.007 CPU Time: 10.0m	P: **0.60993** ± **.067** R: 0.90617 ± .016 AUC: 0.9166 ± 0.010 CPU Time: 12.7m	P: 0.58345 ± 0.072 R: 0.91650 ± 0.018 AUC: **0.9192** ± **0.008** CPU Time: 15.2m	P: 0.56106 ± 0.073 R: **0.92350** ± **0.017** AUC: 0.9181 ± 0.009 CPU Time: 19.5m
3 q layers	P: 0.55618 ± 0.071 R: 0.92334 ± 0.016 AUC: 0.9158 ± 0.013 CPU Time: 17.5m	P: 0.56616 ± 0.069 R: 0.92211 ± 0.016 AUC: 0.9145 ± 0.012 CPU Time: 18.6m	P: 0.55916 ± 0.068 R: 0.92350 ± 0.015 AUC: 0.9147 ± 0.013 CPU Time: 20.1m	P: 0.55315 ± 0.070 R: **0.92382** ± **0.016** AUC: 0.9140 ± 0.012 CPU Time: 23.2m	P: **0.59161** ± **0.062** R: 0.91854 ± 0.016 AUC: **0.9162** ± **0.010** CPU Time: 28.5m
5 q layers	P: **0.58582** ± **0.081** R: 0.91602 ± 0.017 AUC: **0.9162** ± **0.012** CPU Time: 31.7m	P: 0.57983 ± 0.070 R: 0.91772 ± 0.017 AUC: 0.9156 ± 0.011 CPU Time: 35.2m	P: 0.57719 ± 0.076 R: **0.91773** ± **0.017** AUC: 0.9134 ± 0.013 CPU Time: 39.7m	P: 0.58245 ± 0.072 R: 0.91455 ± 0.018 AUC: 0.9158 ± 0.010 CPU Time: 42.2m	P: 0.56230 ± 0.070 R: 0.91748 ± 0.015 AUC: 0.9134 ± 0.012 CPU Time: 48.6m

Table 2: Average ± standard deviation precision, recall, PR-AUC, and total CPU time (minutes) over 100 runs for all **10 qubit** QNN experiments

4.3 Hybrid QNN (15 qubits): gains with shallow depth, trade-offs at high pairing

With *15 qubits* and one quantum layer (Table 3), the model achieves its strongest overall PR-AUC at 10 pairs (0.9229±0.006 over 20 runs) and its highest recall at the same setting (0.9301±0.011). Pushing to very large entanglement budgets (80–135 pairs) raises precision (e.g., ∼0.63) but notably reduces recall and PR-AUC, indicating over-entanglement/over-mixing can hurt minority detection.

With five quantum layers at 15 qubits, recall becomes extremely high (0.956–0.971), but precision collapses (0.27–0.31) and PR-AUC does not improve beyond the 1-layer case (best 0.9205±0.006 at 0 pairs). As with 10 qubits, deeper circuits do not translate into better PR-AUC under our training budget.

4.4 Model size and compute parity

Table 4 shows the two models are capacity–matched: they share the same $(1, 30) \to (1, 30) \to (1, 15) \to (1, 1)$ layout, and their parameter counts differ by less than 1%

15 qubits	0 pairs	10 pairs	30 pairs	80 pairs	135 pairs
1 q layer	P: 0.58691 ± 0.073 R: 0.92359 ± 0.012 AUC: 0.9221 ± 0.004 CPU Time: 6.1h	P: 0.56936 ± 0.069 R: **0.93008** ± **0.011** AUC: **0.9229** ± **0.006** CPU Time: 6.5h	P: 0.58618 ± 0.070 R: 0.92033 ± 0.008 AUC: 0.9215 ± 0.007 CPU Time: 6.8h	P: **0.63109** ± **0.060** R: 0.90122 ± 0.015 AUC: 0.9171 ± 0.008 CPU Time: 7.7h	P: 0.63042 ± 0.072 R: 0.90569 ± 0.013 AUC: 0.9190 ± 0.008 CPU Time: 8.9h
5 q layers	P: 0.27097 ± 0.041 R: 0.96953 ± 0.006 AUC: **0.9205** ± **0.006** CPU Time: 10h	P: 0.27452 ± 0.041 R: **0.97115** ± **0.005** AUC: 0.9143 ± 0.013 CPU Time: 10.7h	P: 0.31151 ± 0.045 R: 0.95689 ± 0.006 AUC: 0.9137 ± 0.009 CPU Time: 11.8h	P: 0.30291 ± 0.037 R: 0.95567 ± 0.008 AUC: 0.9112 ± 0.013 CPU Time: 12.9h	P: **0.31412** ± **0.057** R: 0.95609 ± 0.010 AUC: 0.9073 ± 0.013 CPU Time: 14.8h

Table 3: Average ± standard deviation precision, recall, PR-AUC, and total CPU time (hours) over 20 runs for all **15 qubit** QNN experiments

(2,326 vs. 2,341). The only change is the middle block—quantum TorchLayer in the hybrid versus a dense layer in the classical model. The *Mult-Adds* for the hybrid reflect host-side tensor ops around the quantum interface and do not include the cost of simulating the circuit; true runtime scales with qubit count Q, the number of two-qubit couplings, and quantum depth L_q, so the wall-clock in Tables 2 and 3 is the relevant compute metric. With capacity held constant, any metric differences are attributable to the variational quantum layer (re-uploading + priority entanglement), not extra parameters or a deeper classical stack.

Layer	Hybrid QNN			Classical		
	Out shape	Params	Mult-Adds	Out shape	Params	Mult-Adds
Linear	(1,30)	930	930	(1,30)	930	930
ReLU	(1,30)	0		(1,30)	0	
Linear	(1,30)	930	930	(1,30)	930	930
Tanh	(1,30)	0		(1,30)	0	
TorchLayer / QLayer	(1,15)	450	6750	(1,15)	465	465
Identity / Tanh	(1,15)	0		(1,15)	0	
Linear (output)	(1,1)	16	16	(1,1)	16	16
Totals		**2,326**	**0.01M**		**2,341**	**0.00M**

Table 4: Model summary comparison between the QNN and the classical neural network

4.5 Head-to-head summary and effect sizes

Against the best classical PR-AUC (logistic: 0.9150±0.014), the best hybrid settings improve PR-AUC by *+0.0042* absolute (10q, 1 layer, 80 pairs: 0.9192±0.008) and by *+0.0079* absolute (15q, 1 layer, 10 pairs: 0.9229±0.006, averaged over 20 runs). Relative to the best classical recall (NN: 0.8832±0.014), the best hybrid recall increases by *+0.0468* absolute (15q, 1 layer, 10 pairs: 0.9301±0.011). These gains coincide with a precision–recall trade-off: hybrid settings tuned for recall typically exhibit lower precision than the classical leaders.

4.6 Runtime & Hardware

Computation grows with both depth and coupling budget. For 10 qubits, total training time per configuration rises from ∼9–10 minutes (few pairs, 1 layer) to ∼49 minutes (135 pairs, 5 layers). For 15 qubits, even shallow circuits require hours (6–9h for 1 layer; 10–15h for 5 layers). These costs motivated averaging 20 runs at 15 qubits versus 100 at 10 qubits. All runs for all models were conducted on an Intel Xeon Gold 6254 processor with 64GB of RAM.

Due to computational limitations, a note of caution is required regarding the run counts. The summaries for the 15-qubit model are derived from 20 runs, compared to 100 runs for the 10-qubit and classical models. We therefore interpret the promising gains from the 15-qubit model with correspondingly greater caution, as they exhibit wider statistical uncertainty.

5 Discussion

This study demonstrates that, under matched capacity (Table 4), replacing one dense layer with a variational quantum layer yields small but reproducible gains concentrated in recall and PR-AUC (Tables 2–3; cf. Table 1). The mechanism appears straightforward. Data re-uploading lets a fixed qubit budget see all 30 features, while the correlation-guided pairing schedule mixes empirically dependent features early and then hands off to a global entangling block. This combination moves the operating curve in the direction that matters for fraud detection: more true positives at comparable overall area under the precision-recall curve. The gains peak at moderate entanglement budgets (roughly 10–80 priority pairs); pushing to very large budgets (80–135) tends to over-mix and can reduce recall/PR-AUC, especially at 15 qubits, which aligns with known trainability headwinds as entanglement and effective depth grow (McClean et al., 2018; Ortiz Marrero et al., 2021; Cerezo et al., 2022). The average recall improvement of roughly 4–5% compared to the best classical baseline (from 0.883 to 0.93) illustrates a practically meaningful boost in minority-class detection without inflating model complexity.

Depth shows a similar trade-off: moving from one to five quantum layers inflates recall but sharply depresses precision and does not improve PR-AUC within our training budget, suggesting that shallow, structured circuits are the efficient frontier here. This reinforces that shallow, correlation-aware ansatze can outperform deeper, randomly entangled circuits when trainability and gradient variance are limiting factors. On compute, the Mult-Adds reported beside the hybrid's quantum block are not a faithful proxy for cost; wall-clock is driven by circuit width, number of two-qubit couplings, and depth, which is why the timing columns in Tables 2 and 3 are the relevant indicator. Practically, a compact qubit count with a trimmed, hardware-aware pairing schedule offers the best accuracy-latency trade-off, and any deployment should calibrate thresholds against a business cost curve to manage false positives while preserving the hybrid model's recall advantage. This demonstrates that correlation-guided hybrid layers can achieve superior accuracy–latency trade-offs without exceeding realistic hardware budgets, making them promising for near-term deployment.

The 15-qubit, 3-layer configuration was excluded because training repeatedly collapsed to a degenerate solution: recall saturated near 1.0 while precision plunged and PR-AUC fell below the 1-layer baseline. The extra depth and coupling appear to aggravate vanishing-gradient/barren-plateau effects under our fixed epochs and learning rate, pushing the classifier toward an "always positive" decision boundary (McClean et al., 2018; Ortiz Marrero et al., 2021). This behavior reinforces our main finding that shallow, correlation-aware circuits strike the optimal balance; deeper ansätze would likely require identity-preserving initialization, layerwise growth, or locality-based regularization to remain trainable.

By wiring two–qubit gates according to the top-$|\rho_{ij}|$ pairs from the Pearson matrix, the ansatz bakes in an inductive bias that matches the circuit's connectivity to empirical feature dependencies. This alignment narrows the effective search space and amplifies the gradient signal for informative interactions early in training, thereby establishing a form of guided optimization. This guided optimization, in turn, yields higher recall and modest PR-AUC gains. When the pairing budget is pushed too high, however, over–mixing can dilute informative locality and flatten gradients, which explains the precision–recall trade-offs we observe at 80–135 pairs. Conceptually, correlation-aware entanglement functions analogously to attention mechanisms in classical neural networks, focusing representational capacity on statistically informative relationships while avoiding redundant couplings.

6 Conclusion

In conclusion, this unified analysis shows that correlation-guided entanglement provides a physically interpretable, statistically grounded, and computationally efficient path toward practical quantum-enhanced learning. The hybrid model achieves measurable gains in recall and PR–AUC under fixed computational budgets, demonstrating that a data-driven quantum structure can deliver reproducible advantages even on near-term devices. By aligning quantum connectivity with data-driven feature correlations, this approach bridges interpretability, stability, and performance—offering a promising direction for quantum-assisted decision systems in real-world imbalanced domains such as fraud and anomaly detection.

Limitations of this study include fewer repetitions for the 15-qubit settings (20 runs), anonymized PCA-like features that limit interpretability, reliance on majority downsampling rather than synthetic oversampling, and simulation-only results. These are tractable: a paired bootstrap could quantify the significance of PR-AUC and recall lifts against the best classical baseline; hardware-constrained pairing and adaptive (learned) sparsification could reduce over-coupling; cost-sensitive thresholding would align the operating point with real chargeback and review costs; and replication on additional fraud/anomaly datasets would test robustness beyond this benchmark. Future work will extend this framework to hardware-efficient entanglement layouts, bootstrapped significance testing, and dynamic correlation mapping, enabling more interpretable and scalable quantum–classical integration.

Acknowledgments

Yousaf Khaliq gratefully acknowledges the Experian Data Innovation Lab for providing the initial motivation for this study. The authors also thank the reviewers for their constructive comments.

Data and code availability

The Kaggle credit card fraud detection dataset is freely available on Kaggle, at the following link: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud. The code and experimental results associated with this project is available on GitHub at the following link: https://github.com/ykhaliq2/QNNFraudDetection.

Competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

Funding

The work of Donglin Wang was supported in part by the National Science Foundation under Grant No. DGE-2510274.

Author Contributions

Yousaf Khaliq: Conceptualization, data curation, methodology, software, validation, investigation, formal analysis, writing original draft, writing review and editing; Donglin Wang: Conceptualization, supervision, writing review and editing.

References

Machine Learning Group, ULB, Worldline: Credit Card Fraud Detection. Kaggle Dataset. European card transactions over two days in 2013; features anonymized via PCA (2019). https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud Accessed 2025-10-28

Dornadula, V.N., Geetha, S.: Credit card fraud detection using machine learning algorithms. Procedia computer science **165**, 631–641 (2019)

Itoo, F., Meenakshi, Singh, S.: Comparison and analysis of logistic regression, naïve bayes and knn machine learning algorithms for credit card fraud detection. International Journal of Information Technology **13**(4), 1503–1511 (2021)

Qi, R.: An empirical study on credit risk assessment using machine learning: Evidence from the kaggle credit card fraud detection dataset. Journal of Computer, Signal, and System Research **2**(5), 48–64 (2025)

Hearst, M.A., Dumais, S.T., Osuna, E., Platt, J., Scholkopf, B.: Support vector machines. IEEE Intelligent Systems and their applications **13**(4), 18–28 (1998)

Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. Journal of artificial intelligence research **16**, 321–357 (2002)

Benedetti, M., Lloyd, E., Sack, S., Fiorentini, M.: Parameterized quantum circuits as machine learning models. Quantum Science and Technology **4**(4), 043001 (2019) https://doi.org/10.1088/2058-9565/ab4eb5

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., Lloyd, S.: Quantum machine learning. Nature **549**(7671), 195–202 (2017)

Schuld, M., Petruccione, F.: Quantum ensembles of quantum classifiers. Scientific reports **8**(1), 2772 (2018)

Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E., Latorre, J.I.: Data re-uploading for a universal quantum classifier. Quantum **4**, 226 (2020)

Nick, T.G., Campbell, K.M.: Logistic regression. Topics in biostatistics, 273–301 (2007)

Rigatti, S.J.: Random forest. Journal of insurance medicine **47**(1), 31–39 (2017)

Chen, T., He, T., Benesty, M., Khotilovich, V., Tang, Y., Cho, H., Chen, K., Mitchell, R., Cano, I., Zhou, T., *et al.*: Xgboost: extreme gradient boosting. R package version 0.4-2 **1**(4), 1–4 (2015)

Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. nature **323**(6088), 533–536 (1986)

Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press, Cambridge, UK (2010)

Barenco, A., Bennett, C.H., Cleve, R., DiVincenzo, D.P., Margolus, N., Shor, P., Sleator, T., Smolin, J.A., Weinfurter, H.: Elementary gates for quantum computation. Physical Review A **52**(5), 3457–3467 (1995)

Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S.C., Endo, S., Fujii, K., McClean, J.R., Mitarai, K., Yuan, X., Cincio, L., *et al.*: Variational quantum algorithms. Nature Reviews Physics **3**(9), 625–644 (2021)

McClean, J.R., Boixo, S., Smelyanskiy, V.N., Babbush, R., Neven, H.: Barren plateaus in quantum neural network training landscapes. Nature Communications **9** (2018)

Cerezo, M., Sone, A., Volkoff, T., Cincio, L., Coles, P.J.: Cost function dependent barren plateaus in shallow parametrized quantum circuits. Nature communications

12(1), 1791 (2021)

Grossi, M., Ibrahim, N., Radescu, V., Loredo, R., Voigt, K., Altrock, C., Rudnik, A.: Mixed quantum–classical method for fraud detection with quantum feature selection. IEEE Transactions on Quantum Engineering **3**, 1–12 (2022) https://doi.org/10.1109/TQE.2022.3213474

Egger, D.J., Gambella, C., Marecek, J., McFaddin, S., Mevissen, M., Raymond, R., Simonetto, A., Woerner, S., Yndurain, E.: Quantum computing for finance: State-of-the-art and future prospects. IEEE Transactions on Quantum Engineering **1**, 1–24 (2020)

Wilkens, S., Moorhouse, J.: Quantum computing for financial risk measurement. Quantum Information Processing **22**(1), 51 (2023)

Thakkar, S., Kazdaghli, S., Mathur, N., Kerenidis, I., Ferreira–Martins, A.J., Brito, S.: Improved financial forecasting via quantum machine learning. Quantum Machine Intelligence **6**(1), 27 (2024)

Akrom, M.: Quantum support vector machine for classification task: A review. Journal of Multiscale Materials Informatics **1**(2), 1–8 (2024)

Park, J.-E., Quanz, B., Wood, S., Higgins, H., Harishankar, R.: Practical application improvement to quantum svm: theory to practice. arXiv preprint arXiv:2012.07725 (2020)

Innan, N., Sawaika, A., Dhor, A., Dutta, S., Thota, S., Gokal, H., Patel, N., Khan, M.A.-Z., Theodonis, I., Bennai, M.: Financial fraud detection using quantum graph neural networks. Quantum Machine Intelligence **6**(1), 7 (2024)

Ortiz Marrero, C., Kieferová, M., Wiebe, N.: Entanglement-induced barren plateaus. PRX Quantum **2**, 040316 (2021) https://doi.org/10.1103/PRXQuantum.2.040316

Cerezo, M., Verdon, G., Huang, H.-Y., Cincio, L., Coles, P.J.: Challenges and opportunities in quantum machine learning. Nature computational science **2**(9), 567–576 (2022)

Caro, M.C., Huang, H.-Y., Cerezo, M., Sharma, K., Sornborger, A., Cincio, L., Coles, P.J.: Generalization in quantum machine learning from few training data. Nature communications **13**(1), 4919 (2022)

Huang, H.-Y., Broughton, M., Mohseni, M., Babbush, R., Boixo, S., Neven, H., McClean, J.R.: Power of data in quantum machine learning. Nature communications **12**(1), 2631 (2021)

Abdi, H., Williams, L.J.: Principal component analysis. Wiley interdisciplinary reviews: computational statistics **2**(4), 433–459 (2010)

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., *et al.*: Scikit-learn: Machine learning in python. the Journal of machine Learning research **12**, 2825–2830 (2011)

Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M.S., Alonso-Linaje, G., AkashNarayanan, B., Asadi, A., et al.: Pennylane: Automatic differentiation of hybrid quantum-classical computations. arXiv preprint arXiv:1811.04968 (2018)

Kinga, D., Adam, J.B., *et al.*: A method for stochastic optimization. In: International Conference on Learning Representations (ICLR), vol. 5 (2015). California;